

Digital Circuits

Exercise 2 - Basic Arithmetic and Status Flags

Group/Team No.: G1 T3

Date: December 1st, 2025

Name: Luciano

Name:

Last Name: Carricart

Last Name:

IM No.: 2782740

IM No.:

Lab Exercise 2	max score	score reached
P1.1 Full adder cell	2	
P1.2 Ripple-Carry-Adder	5	
P1.3 Constraints Adder	4	
P2.1 Design file Addsub	5	
P2.2 Testbench Addsub	4	
P2.3 Constraint Addsub	3	
T1.1 FPGA mapping Adder	3	
T1.2 Timing analysis	2	
T2.1 FPGA mapping Addsub	3	
T2.2 Timing analysis	1	
Sum	32	
Report	15	
Penalty		
Total points	47	

Contents

1	Part 1 - Digital Adder	2
1.1	Objective	2
1.2	Behavioral Simulation	2
1.3	Results	2
1.4	Conclusion	3
2	Part 2 - Subtractor and Status Bits	5
2.1	Objectives	5
2.2	Behavioral Simulation	5
2.3	Results	5
2.4	Conclusion	7
3	Appendix	8
3.1	Appendix A - Loading Bitstream vs Flashing	8
3.2	Appendix B - Overflow Flag	8
3.3	Appendix C - Vivado Bitstream Generation/Compilation Errors	9
4	Literature	10

1 Part 1 - Digital Adder

1.1 Objective

Theoretically, the objective of this task is the implementation of an 8-bit ripple carry adder via full adder cells. The adder would be able process two 8-bit input numbers A and B, plus a carry-input bit, resulting in the sum S and an output-carry bit.

Practically, the objective is to load the bitstream to a MODSYS 2.00 Board, XC7A100, connected to two IOM Boards where the user can turn switches (A, B, and c_i inputs) and see LEDs turn on (S and c_o outputs). In order to achieve this, a constraints file has to be modified accordingly.

1.2 Behavioral Simulation

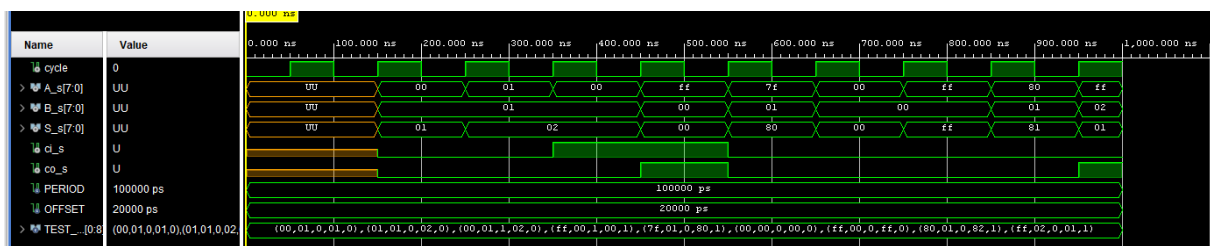


Figure 1: Ripple carry adder with input bit

1.3 Results

Using the Vivado "Report Timing Summary", the propagation delays were inspected for both the synthesis and the implementation of the ripple carry adder design. This was achieved by looking at the unconstrained setup paths.

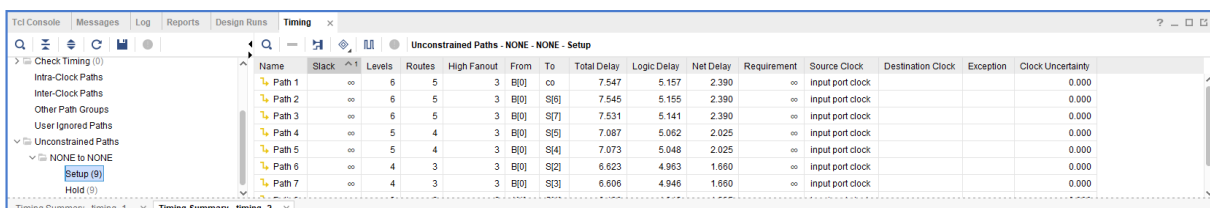


Figure 2: Synthesis Timing Summary

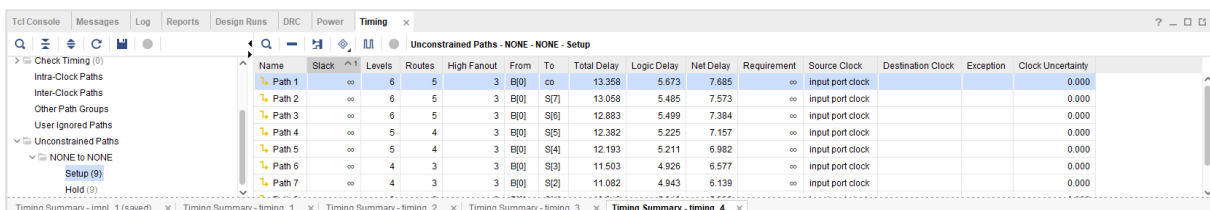


Figure 3: Implementation Timing Summary

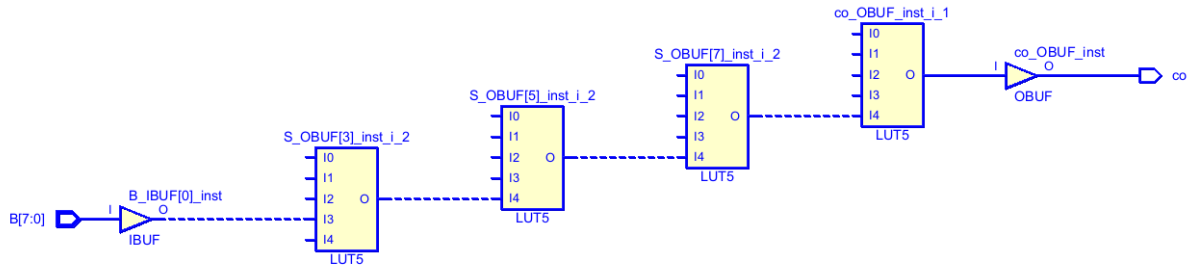


Figure 4: Isolated Path Synthesized Schematic

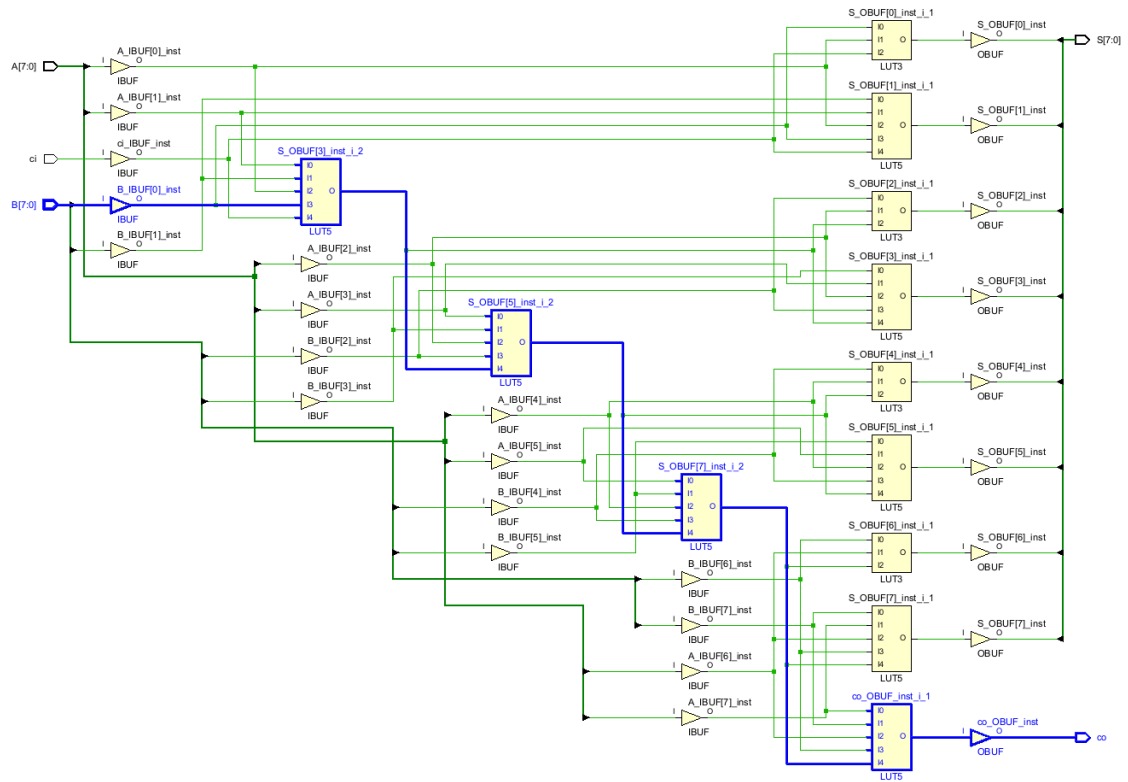


Figure 5: Top-Level Synthesized Schematic

1.4 Conclusion

Looking at the isolated synthesized schematic in Figure 4, one can understand the required components needed to run a specific path in our application, specifically the **critical path**. On the other hand, by looking at the top-level synthesized schematic in Figure 5, one can understand that the critical path will be partially connected to the geographical position of the components needed to achieve the task.

It is also visible to see differences between the timing summary of the synthesized and implemented designs. The critical path of the synthesized design expects a total delay of 7.547 nano seconds, while the implemented design results in 13.358.

In order to understand these differences, it is important to differentiate the terms logic delay (due to components delay) from net delay (due to wiring, where geographical location matters).

The post-synthesis timing report accurately predicts logic delay because it relies on the fixed, pre-characterized timing models of the component libraries. However, the net delay estimation is significantly lower (approx. 3x) than the final implementation. This discrepancy occurs because the synthesis tool cannot account for the physical placement and routing of components on the XC7A100. It lacks the data on real wire lengths and switch matrix usage that is only available after the implementation stage.

2 Part 2 - Subtractor and Status Bits

2.1 Objectives

The objectice of this exercise is to design a ripple carry subtractor/adder that is user-controlled through the use of a *selector* (pressed button). The inputs match the ones of the previous exercise, but the output now includes status bits such as C (carry flag), V (overflow flag), N (negative flag), and Z (zero flag). This is to be implemented in a MODSYS 2.00 Board, XC7A100.

2.2 Behavioral Simulation

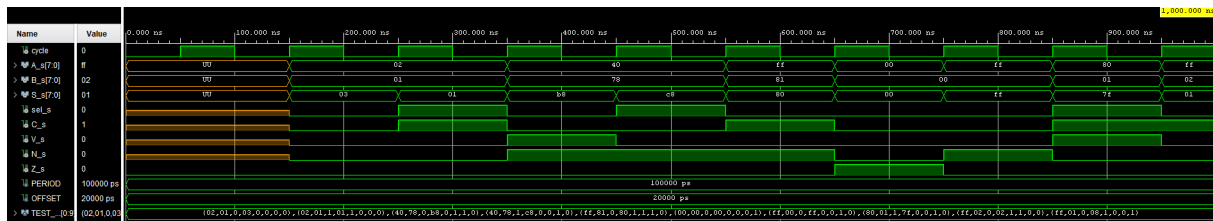


Figure 6: Ripple carry subtractor/adder with input bit

2.3 Results

Using the Vivado "Report Timing Summary", the propagation delays were inspected for both the synthesis and the implementation of the ripple carry adder/subtractor design. This was achieved by looking at the unconstrained setup paths.

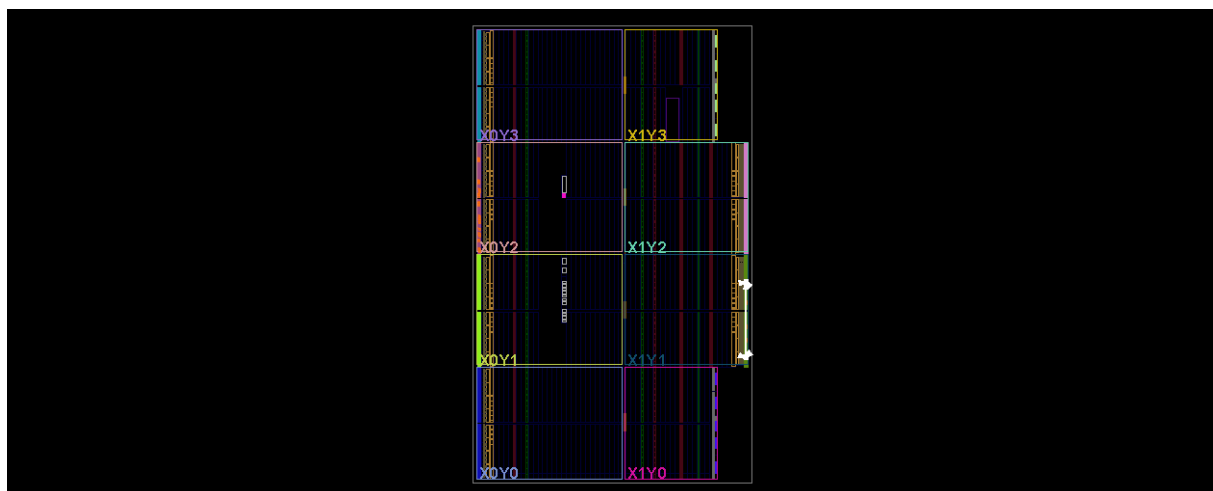


Figure 7: Physical Synthesis of a specific path

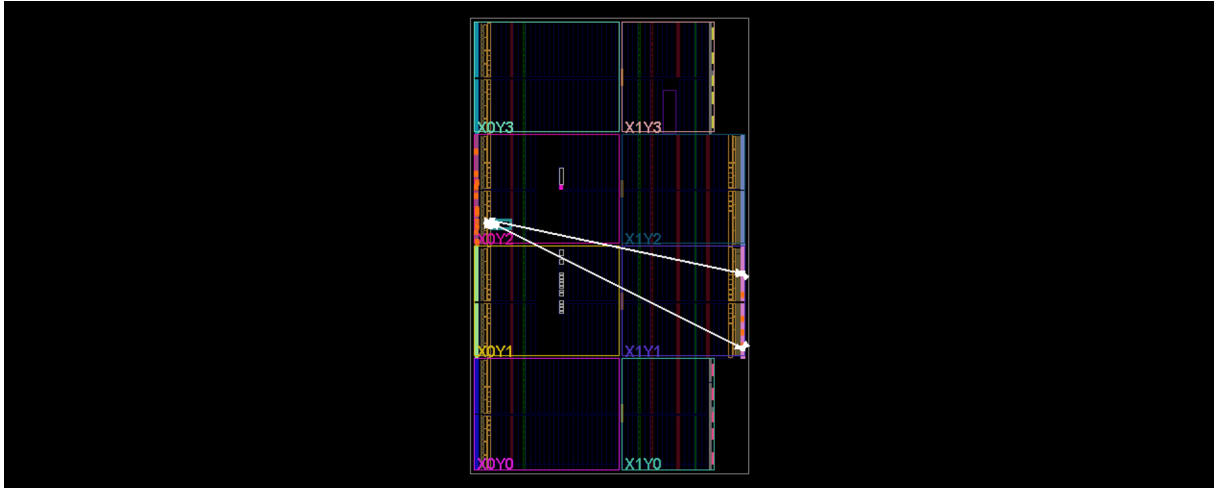


Figure 8: Physical Implementation of a specific path

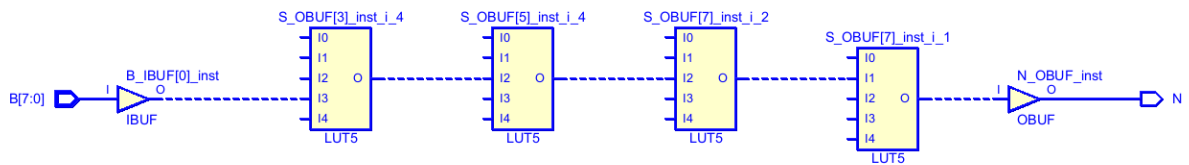


Figure 9: Synthesized Schematic of a specific path

Timing														
Unconstrained Paths - NONE - NONE - Setup														
Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 1	∞	6	5	10	B[0]	Z	7.979	5.168	2.811	∞	input port clock			0.000
Path 2	∞	6	5	10	B[0]	V	7.847	5.226	2.622	∞	input port clock			0.000
Path 3	∞	6	5	10	B[0]	C	7.819	5.197	2.622	∞	input port clock			0.000
Path 4	∞	6	5	10	B[0]	S[6]	7.817	5.195	2.622	∞	input port clock			0.000
Path 5	∞	6	5	10	B[0]	S[7]	7.803	5.181	2.622	∞	input port clock			0.000
Path 6	∞	6	5	10	B[0]	N	7.793	5.171	2.622	∞	input port clock			0.000
Path 7	∞	5	4	10	B[0]	S[5]	7.348	5.082	2.266	∞	input port clock			0.000

Figure 10: Synthesized Timing Summary

Timing														
Unconstrained Paths - NONE - NONE - Setup														
Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 1	∞	6	5	10	B[0]	N	16.338	5.455	10.883	∞	input port clock			0.000
Path 2	∞	6	5	10	B[0]	S[6]	14.478	5.659	8.819	∞	input port clock			0.000
Path 3	∞	6	5	10	B[0]	S[7]	13.892	5.465	8.427	∞	input port clock			0.000
Path 4	∞	6	5	10	B[0]	C	13.775	5.667	8.108	∞	input port clock			0.000
Path 5	∞	6	5	10	B[0]	Z	13.614	5.492	8.122	∞	input port clock			0.000
Path 6	∞	6	5	10	B[0]	V	13.572	5.510	8.062	∞	input port clock			0.000
Path 7	∞	5	4	10	B[0]	S[5]	13.186	5.214	7.972	∞	input port clock			0.000

Figure 11: Implemented Timing Summary

2.4 Conclusion

Interesting conclusions can be reached by looking at the physical designs –such as the ones shown in Figures 8 and 7– and the schematic design. The physical designs for a specific path show one starting point and two ending points, whereas the schematic shows a single starting point and a single ending point. The add/sub design introduces control signals (invert B, select add vs subtract, multiplexing) that can increase fanout or create additional sinks. A net with fanout produces a larger routed tree; the worst branch to the path endpoint can be much slower than a point-to-point net. This is consistent with the “two roads” behavior.

On the other hand, it is visible how the total delays of the timing summaries of the synthesis (Figure 10) and implementation (Figure 11) differ significantly in this application. While the logic delays are borderline equal in both summaries, the net delays for the same path (B[0] to N) has a difference of around 5x. This difference is much higher than the one shown in the Adder application (around 3x), and it could be caused due to the “two roads” behavior seen in Figure 8, not present in the previous Adder application.

3 Appendix

3.1 Appendix A - Loading Bitstream vs Flashing

There is a difference between the terms "load the bitstream" and "flash the device".

- Loading the bitstream implies writing the configuration directly to the FPGA's SRAM (volatile memory) via JTAG. The result is a device that works immediately, but if you unplug the power, the configuration is lost.
- Flashing the device implies writing the configuration to a Non-Volatile Memory chip (like an SPI Flash or EEPROM) that lives on the board (or inside the FPGA if it's a flash-based FPGA). The result is configuration that survives a power cycle; when the device turns on, it automatically loads itself from this flash memory.

3.2 Appendix B - Overflow Flag

The Overflow Flag (OF) is a status flag in the processor's condition code register that indicates when a signed arithmetic operation has produced a result that is too large (or too small) to fit into the destination operand's bit width.

In signed arithmetic (Two's Complement), the Most Significant Bit (MSB) acts as the sign bit (0 for positive, 1 for negative). An overflow occurs when the addition of two numbers with the *same sign* results in a value with the *opposite sign*.

Consider an 8-bit signed addition operation.

- **0xFF**: In Two's Complement, this represents -1 .
- **0x80**: In Two's Complement, this represents -128 .

Mathematically, we expect: $(-1) + (-128) = -129$. However, an 8-bit signed integer can only represent values between -128 and $+127$. The result -129 is outside this range. Let us observe the binary addition:

```
  1111 1111   (0xFF or -1)
+ 1000 0000   (0x80 or -128)
-----
1 0111 1111   (Result in 8-bit register: 0x7F)
~
(Carry Out)
```

Analysis of the Result:

1. The processor truncates the result to 8 bits: 0111 1111 (0x7F).
2. The MSB of the result is **0**, which indicates a **positive** number ($+127$).
3. We added two **negative** numbers (MSB 1) and obtained a **positive** result (MSB 0).

Because it is impossible for two negative numbers to sum to a positive number, the processor sets the **Overflow Flag (OF = 1)** to signal that the result in the register ($+127$) is mathematically incorrect relative to the signed operands.

3.3 Appendix C - Vivado Bitstream Generation/Compilation Errors

In order to avoid compiling errors in future Vivado compiling/bitstream generations, a few lessons learned are highlighted:

- If the programmer doesn't use a port <that is defined in design file!> in the physical solution, then the program will not simulate correctly. The application will compile, but the Behavioral Simulation will not show accurate results. The fact that the compilation works makes the problem very hard to spot. In summary, the ports (used / not used) should match in all *Design File*, *Test Bench*, and *Constraint File*.

4 Literature

[1] J. F. Wakerly, Digital Design: Principles and Practices, 5th ed. Upper Saddle River, NJ, USA: Pearson, 2018.